Code review — online-school-vanilla-java

Runde de review pe proiectul tău, Adrian, cea mai nouă sus.

Review pe proiectul online-school. Runda cea mai nouă e sus.

Runda 5 — 2026-10-28 — sha ce8b172

Ce e nou: ierarhia `User` / `Teacher` / `Student` + `UserRepository` (`8329be1`), Strategy pe meniu (`cbc84a6`) și Observer pe înscrieri (`ce8b172` , exercițiul din `exercitiu-observer-inscrieri.pdf`). Poarta de compilare: `javac` trece pe toate sursele, 0 erori. Totul de mai jos e reprodus prin rulare, pe o copie a proiectului. Fișierele tale n-au fost atinse.

Observer: ce ai făcut bine și ce lipsește

Scheletul e corect, și asta e partea grea:

- `EnrollmentRepository` cunoaște doar `ObservatorInscriere` , prin `abonare` / `dezabonare` și o listă. Nu există niciun `new JurnalInscrieri` sau `NotificareProfesor` în el.
- În `addEnrollment` (`:46–58`) verifici duplicatul **înainte** de `add` și anunți **după** `save()` . De aceea, la a doua înscriere la același curs, apare `You are already registered for this course.` și niciun observator nu scrie nimic. Asta e exact întrebarea „De ce?" din fișă, iar codul tău îi răspunde corect.
- `NotificareProfesor` primește `CourseRepository` și `UserRepository` în constructor și nu le creează singur. Abonarea se face o singură dată, în `View` .
- La Strategy, `View.play()` nu mai știe nimic despre roluri: alegerea se face într-un singur loc, în `alegeMeniu()` . Comentariul tău de la `View.java:48` despre shadowing arată că ai înțeles de ce `MeniuRol meniuRol = ...` era greșit.

Ce nu merge încă: profesorul nu e notificat niciodată (B13, B15), testul din fișă nu se poate rula din meniu (B9), iar mesajele nu sunt cele din cerință (M15–M17).

Rundele anterioare

#	Runda 4	Acum
B9	opțiunea 7 cheamă `removeCourse`	🔴 PARȚIAL: `registerForCourse` e legată pe 8, dar 7 tot dezînscrie, și asta sub eticheta „inscrie-te". Vezi B9 mai jos
B10	`updateBookName` fără `save()`	🔴 NEATINS: redenumesc „Domain-Driven Design" în „DDD 2", iar după repornire numele vechi e înapoi
B2	unicitatea la `update()` se poate ocoli	🔴 NEATINS, a treia oară: `UserRepository.update` (`:82–87`) lasă doi utilizatori cu `andrei.ionescu@example.com`, iar `CourseRepository.update` (`:102–110`) lasă două cursuri „Java Programming"
B6	un student șterge cartea altuia	🔴 NEATINS: Gabriela șterge „Refactoring" (a lui `c8f–45–71`) și primește `success`
B7	`abc` la meniu omoară aplicația	🔴 NEATINS: `NumberFormatException` la `View.java:69`
B11	renunțare la un curs neînscris, stack trace	🔴 NEATINS: `NoSuchElementException` la `View.java:138`
B12	`cardRepository.save()` scrie `toString()`	🔴 NEATINS: `cardRepository.java:63` , ar scrie `CardStudent@49476842`
B8	linie cu 3 câmpuri, aplicația nu pornește	🔴 NEATINS: `Book.java:18` , `Index 3 out of bounds for`

#	Runda 4	Acum
		`length 3`
M13	duplicate în `enrollments.txt`	neatins: după „renunță la Web Development", Gabriela e tot înscrisă (`C003` apare de două ori)
M6, M7, M8, M10–M12, M14		neatinse (M7 și M8 se văd în fiecare stack trace de mai jos)
C3, C4, C8, C9, C12, M5		neatinse

💡 Opt 🔴 din runda 4 sunt tot acolo. B10 și B12 se repară fiecare cu un singur cuvânt. Recitește runda 4 (e păstrată mai jos) înainte de pattern-ul următor.

🔴 Critice

B13 🔴 `UserRepository` nu mai găsește fișierul: 0 utilizatori, fără nicio eroare

`src/app/users/repository/UserRepository.java:18`

CUM E ACUM:

```
new UserRepository().count()  ->  0
findById("9755fd14-...")      ->  Optional.empty   (profesorul de la C001)
add(...)                      ->  IllegalStateException:
    Cannot write file: src/app/student/data/users.txt
```

DE CE: În `ce8b172` ai redenumit pachetul `student` în `users`, iar IntelliJ a mutat și `users.txt` în `src/app/users/data/`. `FILE_PATH` a rămas însă `"src/app/student/data/users.txt"`. Rename-ul schimbă numele de pachet (`app.student.repository` , cu puncte), dar o cale cu `/` dintr-un String e pentru el text oarecare. `loadData()` face `if (!file.exists()) return;` , deci un fișier lipsă arată exact ca un fișier gol. De aici vine tăcerea: profesorul nu e găsit, iar observatorul nu afișează nimic.

💡 Ce ar trebui să facă `loadData()` când fișierul nu există: să tacă sau să spună? Uită-te la `EnrollmentRepository` , `CourseRepository` și `BookRepository` : toate au același `return` .

💡 Caută `"src/app/` în tot proiectul și compară fiecare cale cu folderul real.

B14 🔴 După un `save`, rolul se pierde, iar id-ul se strică

`UserRepository.java:126` · `Student.java:22–23` · `Teacher.java:34–35`

CUM E ACUM (cu `users.txt` pus la calea pe care o caută codul):

```
add(new Teacher("x,Ana,Pop,..."))  ->  id = e9abe8db-...
linia scrisa in users.txt:
    TEACHERe9abe8db-...,Ana,Pop,ana.pop@example.com,Pw1,40
dupa repornire  ->  clasa=User  rol=USER  id=TEACHERe9abe8db-...
findById("e9abe8db-...")  ->  false
```

DE CE: Aici sunt două greșeli care se adună. La scriere, `getRol() + super.toText()` lipește rolul de id fără virgulă, așa că rolul devine o bucată din id. La citire, `loadData()` face mereu `new User(line)`, deci nu poate ieși niciodată un `Teacher` sau un `Student`. Nici n-ar avea de unde: în `users.txt` nu există o coloană care să spună rolul. Obiectul își știe clasa doar cât trăiește în memorie. Pe disc ajunge doar ce scrie `toText()`, iar la pornire clasa se reconstruiește doar din ce citește `loadData()`.

💡 Ce coloană ar trebui să aibă fiecare linie din `users.txt` ca `loadData()` să știe ce obiect creează? Unde pui decizia „TEACHER → `new Teacher`”: într-un singur loc sau în fiecare clasă?

💡 Formatul scris de `toText()` și cel citit de constructor trebuie să fie același. Scrie o linie cu `toText()` și dă-o înapoi constructorului. Iese același obiect?

B15 🔴 `NotificareProfesor` crapă cu `ClassCastException`, iar înscrierea rămâne neanunțată

`src/app/enrollment/observers/NotificareProfesor.java:52–53`

CUM E ACUM (cu B13 ocolit, deci cu utilizatorii încărcați), 8 → Java Programming:

```
java.lang.ClassCastException: class app.users.model.User
    cannot be cast to class app.users.model.Teacher
  at ...NotificareProfesor.lambda$scrie$0(NotificareProfesor.java:52)
  at ...EnrollmentRepository.addEnrollment(EnrollmentRepository.java:54)
```

Nu apare nici `enrolled`, nici linia de jurnal, deși înscrierea e deja salvată în `enrollments.txt`.

DE CE: Un cast nu transformă obiectul. Doar verifică, la rulare, că obiectul chiar este un `Teacher`. Din cauza B14, din fișier nu ies decât `User`, deci verificarea pică. Cast-ul nici nu era necesar: `getFirstName()` / `getLastName()` sunt în `User`. Mai e o lecție despre Observer aici: subiectul își

cheamă observatorii unul după altul, în aceeași buclă. Excepția unuia oprește bucla, iar cei abonați după el nu mai află nimic. Asta e și motivul pentru care jurnalul tace: tu l-ai abonat al doilea (vezi M17).

💡 Ce metode folosești de fapt din `prof`? În ce clasă sunt declarate?

💡 Același cast stă și la `View.java:114`, `(Student) user`. Ce se întâmplă acolo dacă utilizatorul logat e profesor?

B9 🔴 RESTANȚĂ, parțial: 7 dezînscrie, iar 8 înscrie și închide aplicația

`src/app/meniu/MeniuStudent.java:23–24, 48–54`

CUM E ACUM, cu scriptul din fișă (7 → Java Programming, 7 → Web Development, 8 → Java Programming, 0). Ieșirea, scurtată:

```
7 Java Programming -> NoSuchElementException: No value present
                      (View.java:138)
7 Web Development  -> 2026-10-28 t8v-16-92 -> C003 renuntare
                      <- trebuia sa te inscrie
8 Java Programming -> 2026-10-28 t8v-16-92 -> C001 inscriere
                      enrolled
                      (aplicatia se inchide, 0 nu mai e citit)
```

Meniul afișează `7-inscrie-te la un curs:` și `8-inscrie-te la un curs`, deci de două ori aceeași etichetă, iar `0` lipsește.

DE CE: Ai mutat `registerForCourse` pe 8, dar ai lăsat 7 pe `removeCourse` și n-ai pus `break` după `case 8`. Fără `break`, `switch` nu se oprește la sfârșitul unui `case` și intră în următorul, adică în `case 0: return false`. `play()` primește `false` și iese din buclă după fiecare înscriere, chiar și după una refuzată ca duplicat. În runda 4, exact asta era cerința: „7 înscrie, 8 renunță, plus 0 în meniu". Fișa observer o repetă la pasul 1.

💡 Citește pasul 1 din fișă și compară-l, rând cu rând, cu `afiseazaOptiuni()` și `executa()`.

💡 În `MeniuTeacher` ai pus `break` peste tot. Ce are în plus `case 8` față de `case 1`?

B16 🔴 `CourseRepository.save()` pierde `teacherId`, iar după repornire aplicația nu mai pornește

`src/app/course/model/Course.java:70–72`

CUM E ACUM (prin cod, pentru că din meniu nu se modifică cursuri încă):

```
update(C001) -> courses.txt: C001,Java Programming,Computer Science
                (3 coloane, teacherId a disparut)
repornire    -> IllegalStateException: Invalid course at line 1:
                C001,Java Programming,Computer Science
```

DE CE: Ai adăugat a 4-a coloană în `courses.txt` și ai pus `arr[3]` în constructor, dar `toText()` scrie tot 3 câmpuri. `save()` rescrie fișierul întreg, deci un singur `update` șterge profesorul de la toate cursurile. Constructorul cere apoi `arr[3]`, iar aplicația nu mai pornește deloc. E aceeași regulă ca la B12: ce scrii trebuie să poată fi citit înapoi. Aici n-ai pierdut câmpul la un model nou, ci la unul pe care l-ai extins.

💡 Pune `toText()` și constructorul unul lângă altul. Numără câmpurile în fiecare.

💡 Când adaugi o coloană într-un `.txt`, care sunt cele două locuri din model pe care le atingi? Verifică-le și la `Book`, `Enrollment`, `CardStudent`.

B17 🔴 Profesorul vede un singur student pe curs

`src/app/View.java:294` (`findEnrollmentByCourseId`)

CUM E ACUM (pe o copie cu `View` pornit ca profesorul `9755fd14`, cu `users.txt` găsit), 2 → Java Programming:

```
1
a5e-12-32,Bogdan,Popescu,bogdan.popescu@example.com,Pass123,22
[app.users.model.User@27c170f0]
```

La C001 sunt înscriși 3 studenți (`a5e-12-32`, `m6n-52-67`, `j1k-74-19`). Fără B13 ocolit, apare `Niciun student nu s-a inscris inca la cursul acesta.` urmat de `null`.

DE CE: `findEnrollmentByCourseId` întoarce un `Optional`, adică cel mult o înscriere: prima din listă cu acel curs. Pentru „toți studenții unui curs" ai nevoie de o listă. Ai scris deja varianta cu listă pentru student, în `findAllEnrollmentsByStudentId`. Ultima linie, `[...User@27c170f0]`, apare pentru că `showStudentsFromCourse` face `println` pe lista întoarsă. Pe `User` nu ai `toString()`, deci se folosește cel din `Object` (tu ai notat asta la `:318`).

💡 Câte înscrieri poate avea un curs? Ce tip ar trebui să întoarcă metoda din repository?

💡 Metoda se numește `show...`, dar întoarce o listă care e apoi și afișată. Alege una dintre ele.

🟡 Importante

- **M15** Mesajele observatorilor nu sunt cele din cerință. Jurnalul afișează `2026-10-28 t8v-16-92 -> C001 inscriere`, iar fișa cere `[2026-10-28] student t8v-16-92 -> curs C001: inscris` (și `renuntat`). `NotificareProfesor:52-53` pune numele de familie înaintea prenumelui și spune „un student nou … retras" și la renunțare. Fișa cere „un student a renuntat la cursul …".
- **M16** Lipsește ramura „nu are profesor". La 8 → Operating Systems (C005, `teacherId` = `""`) apare doar linia de jurnal, iar fișa cere `Cursul Operating Systems nu are profesor de notificat.` Acum `ifPresent` tace, ca la B13.
- **M17** `EnrollmentRepository.deleteById` (`:68-75`) anunță observatorii **înainte** de `remove` + `save()`. Fișa cere anunțul după, ca la `addEnrollment`. Tot în `View.java:44-45` ai abonat întâi profesorul, apoi jurnalul, invers față de fișă. B15 arată de ce contează ordinea.
- **M18** `JurnalInscrieri` primește `CourseRepository` și `UserRepository`, dar nu le folosește. Fișa: „nu are nevoie de nimic în plus".
- **M19** Capcană în date: `courses.txt:10` are un spațiu înainte de `teacherId` (`Business, 9755fd14-...`). Profesorul vede 4 cursuri din 5, pentru că C010 lipsește din `findCoursesByTeacherId`.
- **M9 (actualizat)** Nu există login: `View.java:37` pornește mereu ca Gabriela (student), deci `MeniuTeacher` nu se poate deschide din aplicație. Strategy e făcut corect, dar alegerea depinde de un utilizator scris de mână. `Main.java:24-28` creează un `User`, un `Course` și un `Enrollment` după ce `View` s-a terminat, iar pe niciunul nu-l folosește.
- **M20** Cod mort în jurul rolurilor: `checkUserType`, `checkUserRole` (`View.java:339-354`) și `findUserTypeById` nu sunt chemate de nicăieri. `checkUserRole` întoarce oricum mereu `"user not found"`, pentru că rezultatul de la `:350` e aruncat. Tot acolo, `User.getCoursesList()` (`User.java:85-87`) întoarce `null`, iar `assignCourseToTeacher` nu e chemată.
- **M21** Înscrierea de test a rămas comisă: `enrollments.txt` are linia nouă `t8v-16-92,C008,2026-10-27`. Fișa cerea, la final, să readuci fișierul la starea dinainte.

🟢 Cleanups

- **C13** Pachetul cerut e `app.enrollment.observer`, iar tu ai `observers`.
- **C14** `rol` e declarat în `User`, `Student` și `Teacher`: fiecare clasă are câmpul ei, care îl ascunde pe cel din părinte. Îți ajunge `getRol()` suprascris. Pe lângă asta, `setRol("TEACHER")` pe un `Student` ar produce un student care se declară profesor.
- **C15** Codul comentat `teacherMeniu` / `playTeacher` (`View.java:211-246`) și metoda goală `showCoursesByEnrollmentIds` (`:267`) trebuie șterse. Acum le ține minte git.
- **C16** `CODE_REVIEW.md` din repo l-am pus eu în rundele vechi. De acum primești doar `CODE_REVIEW.pdf`, iar `.md`-ul îl poți scoate (`git rm CODE_REVIEW.md`). Interesant: în `ce8b172`,

rename-ul de pachet a modificat textul din `.md`, dar n-a modificat calea din `UserRepository` (B13).

- Bine: ai redenumit `Meniu` în `meniu` și `student` în `users`, după convenția Java pentru pachete.

Q&A

1. **B13 + M16:** `loadData()` și `NotificareProfesor` au tăcut amândouă când n-au găsit ce căutau. De ce e mai greu de găsit un bug care tace decât unul care aruncă excepție?
2. **B15:** Dacă `NotificareProfesor` aruncă excepție, ce află `JurnalInscrieri`? Dar dacă îi schimbi ordinea la abonare?
3. **B14:** Ce trebuie să conțină o linie din `users.txt` ca, după repornire, `alegeMeniu()` să ajungă la `MeniuTeacher` fără niciun `new Teacher(...)` scris de mână?

Ordinea de lucru

1. **B9**: meniul (7, 8, 0, `break`). Fără el nu poți rula testul din fișă.
2. **B13 → B14 → B15**: utilizatorii se încarcă, cu rolul lor, iar profesorul primește mesajul.
3. **M15–M18**, apoi rulezi scriptul din fișă și compari ieșirea rând cu rând.
4. **B16, B17**, apoi restanțele din tabel. B10 și B12 durează câte un minut.

Runda 4 — 2026-09-09 — sha 1bfb58b

Stare verificată prin rulare pe `ce8b172`.

#	Constatare	Stare
B9 🔴	meniul 7 („înscrie-te”) cheamă `removeCourse()`, iar `registerForCourse()` nu e chemată	🔴 parțial: 8 înscrie, dar fără `break`, iar 7 tot dezînscrie
B10 🔴	`updateBookName` fără `save()`, numele vechi revine după repornire	🔴 neatins
B2 🔴	unicitatea la `update()` se ocolește (`StudentRepository`), iar în `CourseRepository.update` lipsește	🔴 neatins (acum în `UserRepository`)
B6 🔴	`deleteByBookName` șterge cartea altui student	🔴 neatins
B7 🔴	`Integer.parseInt` pe `abc` omoară aplicația	🔴 neatins
B11 🔴	`Optional.get()` pe gol la renunțare, stack trace	🔴 neatins
B12 🔴	`cardRepository.save()` scrie `toString()`, nu `toText()`	🔴 neatins
B8 🔴	`Book` / `Enrollment` / `CardStudent` cer necondiționat `arr[n]`	🔴 neatins
M13	duplicate în `enrollments.txt`, acceptate de `loadData()`	neatins
M6	unicitate globală pe numele cărții	neatins (`Effective Java` : `Book is already in the store.`)
M9	studentul curent hardcodat în `View`	neatins (vezi M9 în runda 5)
M7, M8	`printStackTrace()` la utilizator, `play()` din constructor	neatinse

#	Constatare	Stare
M10, M11, M12, M14	cod orfan, `0` lipsă din meniu, liste care tac, metode duble în `EnrollmentRepository`	neatinse
C10	importuri `Student` rămase în `Book`, `CardStudent`	✅ închis
C9	importuri parazite	parțial: `javax.smartcardio.Card` scos, `javax.swing.text.html.Option` (`BookRepository.java:6`) rămas
C3, C4, C8, C11, C12, M5	excepții I/O, snake_case, `stundentId`, două API-uri de ștergere, `cardRepository`, `Book` în afara `model/`	neatinse

Runda 3 — 2026-08-19 — sha a92b793

Relații între entități peste stratul de repository.

#	Constatare	Stare
B1 🔴	`Main` crapă la orice rulare	✅ închis
B2 🔴	`update()` acceptă duplicate și le salvează	🔴 neatins (vezi runda 5)
B3 🔴	`CourseRepository` nu încarcă datele	✅ închis
B4 🔴	`addBook()` aruncă id-ul primit	✅ închis
B5 🔴	aceeași carte la doi studenți	✅ închis, prin cheie străină pe carte
M1	relațiile nu se persistau	✅ închis
M4	unicitate globală pe numele cărții	neatins (M6)
M5, C3, C4	`Book` în afara `model/`, excepții I/O, snake_case	neatinse
C1	câmpuri de test în `Student`	✅ închis

Rundele 2 și 1

Stratul de repository: `findById` / `findByX` cu `Optional`, `existsByX`, `indexOf` cu `-1`, `save()` care rescrie tot fișierul, `loadData()` cu numărul liniei în eroare. Tiparul a fost înțeles și aplicat de patru ori, singur.